 Run in Google Colab  View on Github  View raw on Github

Module 9: Estimation

In this lab, we will learn about Kernel Density Estimation (KDE), interpolation, and (briefly) regression.

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import altair as alt
import pandas as pd
import scipy.stats as ss
%matplotlib inline
```

Kernel density estimation

Let's import the IMDb data.

```
import vega_datasets

movies = vega_datasets.data.movies()
movies.head()
```

	Title	US_Gross	Worldwide_Gross	US_DVD_Sales	Production_Budget	Release_
0	The Land Girls	146083.0	146083.0	NaN	8000000.0	Jun 12
1	First Love, Last Rites	10876.0	10876.0	NaN	300000.0	Aug 07
2	I Married a Strange Person	203134.0	203134.0	NaN	250000.0	Aug 28
3	Let's Talk About Sex	373615.0	373615.0	NaN	300000.0	Sep 11
4	Slam	1009819.0	1087521.0	NaN	1000000.0	Oct 09

Although we have learned that it is dangerous to drop any missing values, we will do so for the sake of simplicity. We are also really not trying to draw any conclusions about the data so it is okay. But be careful with missing data in practice!

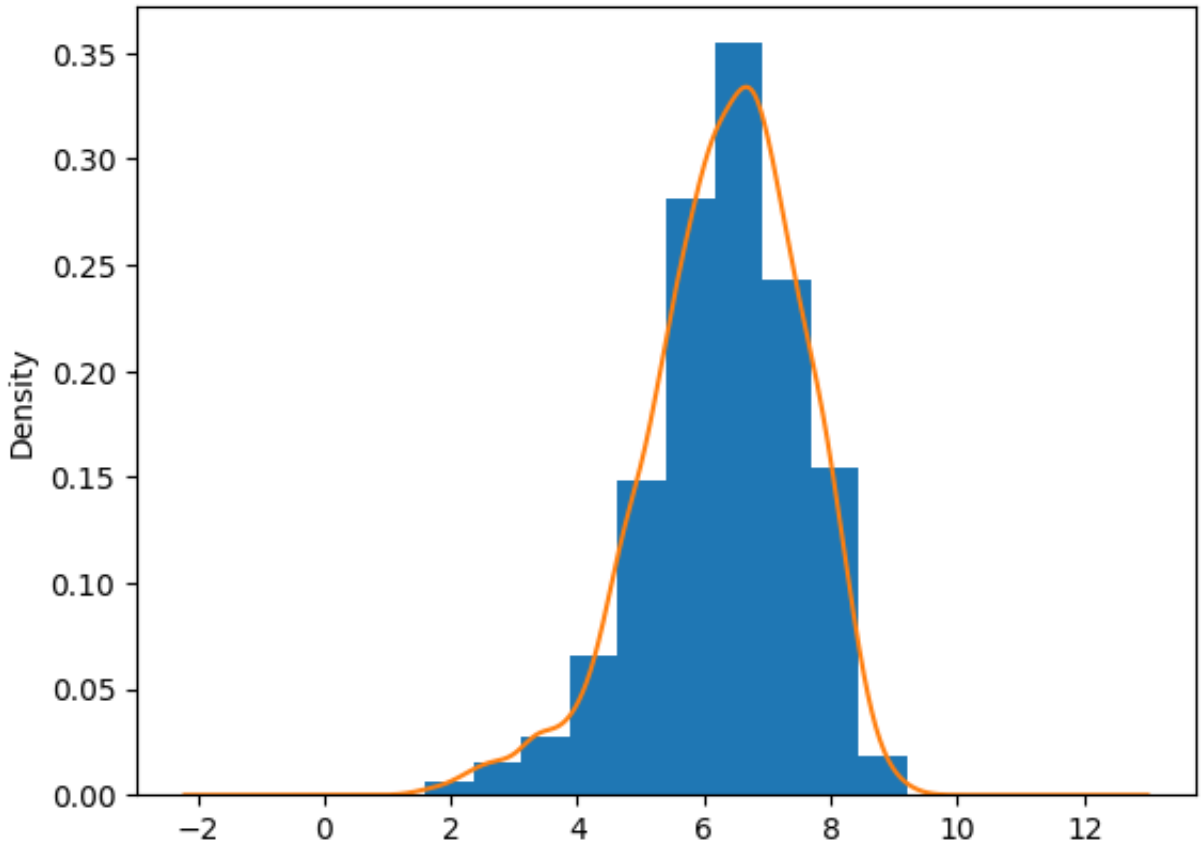
Q: Can you drop rows that have NaN value in either `IMDB_Rating` or `Rotten_Tomatoes_Rating`?

```
# YOUR SOLUTION HERE
movies = movies.dropna(subset=['IMDB_Rating', 'Rotten_Tomatoes_Rating'])
```

We can plot histogram and KDE using pandas:

```
movies['IMDB_Rating'].hist(bins=10, density=True)
movies['IMDB_Rating'].plot(kind='kde')
```

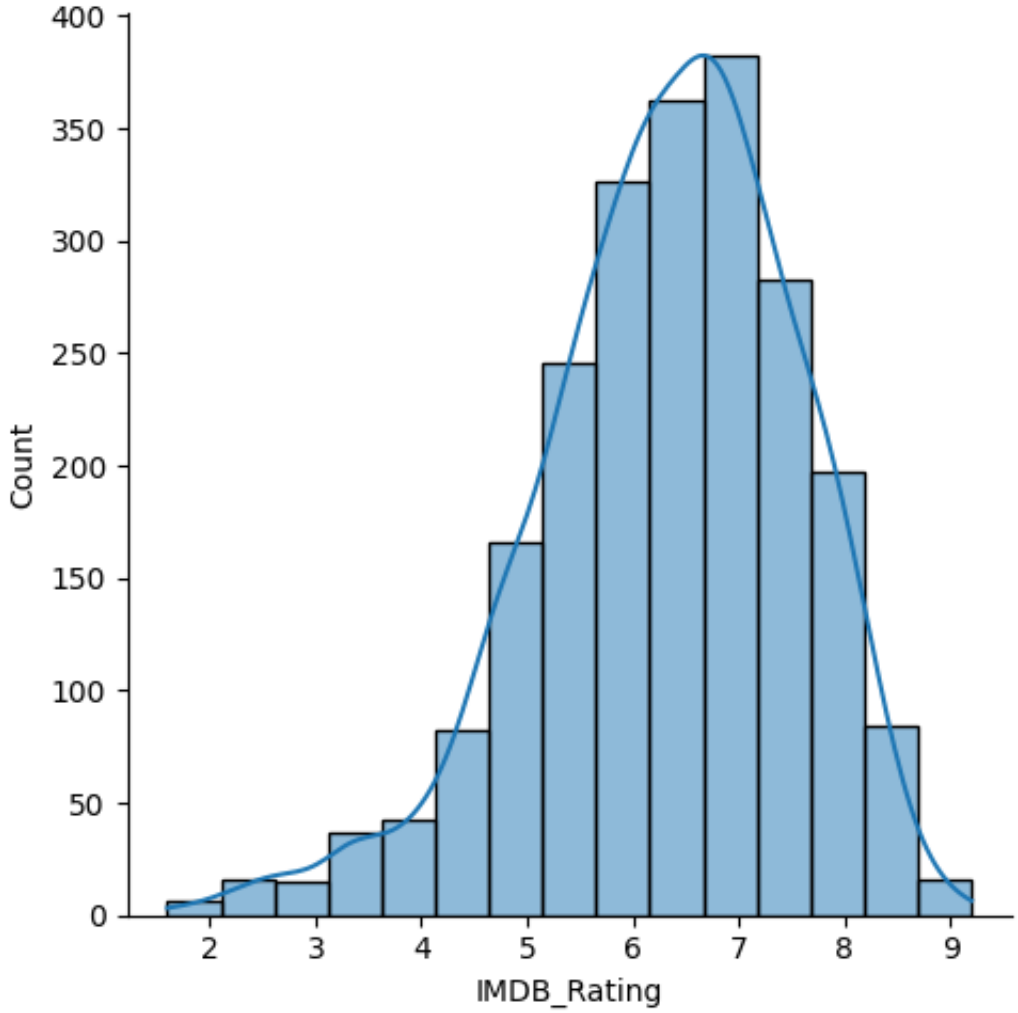
```
<Axes: ylabel='Density'>
```



Or using seaborn (two ways):

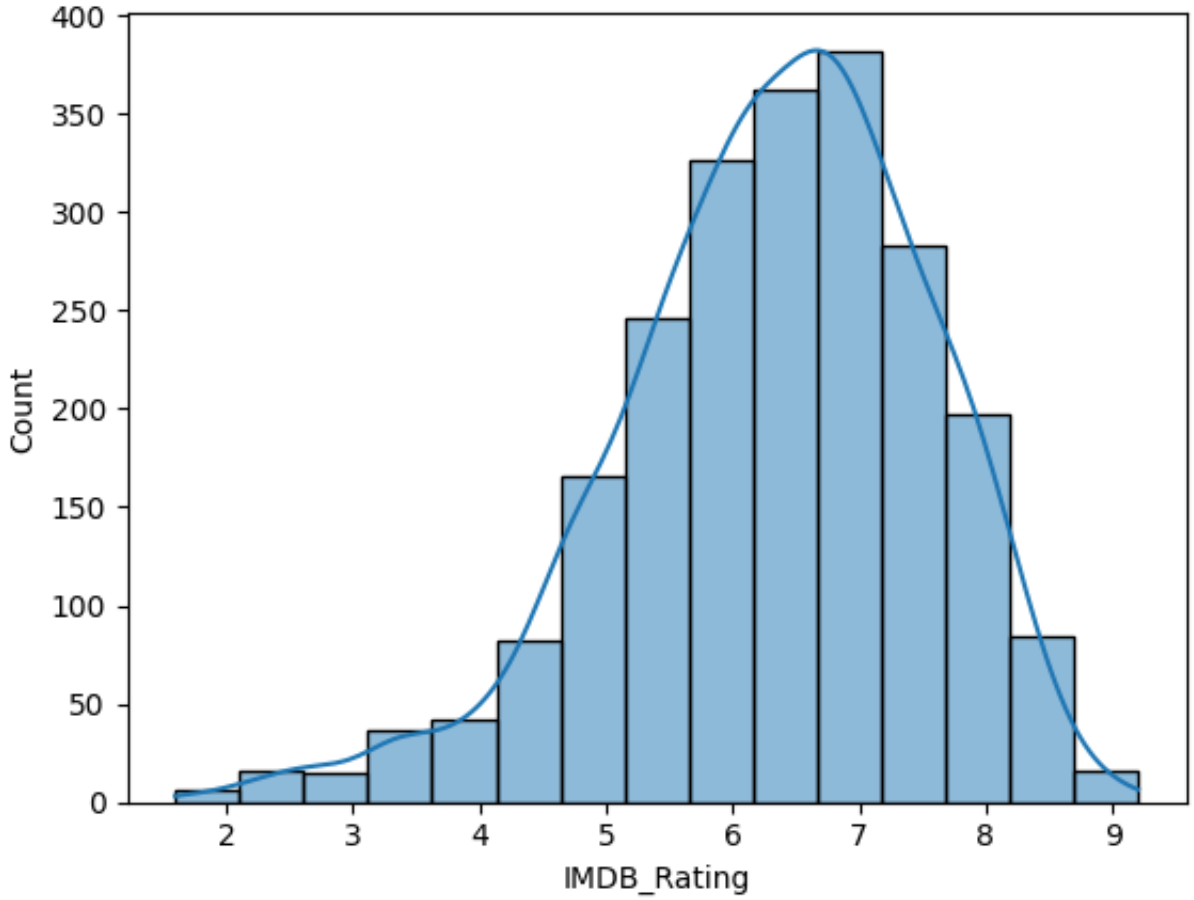
```
sns.displot(movies['IMDB_Rating'], bins=15, kde=True)
```

```
<seaborn.axisgrid.FacetGrid at 0x12bdd1580>
```



```
sns.histplot(movies['IMDB_Rating'], bins=15, kde=True)
```

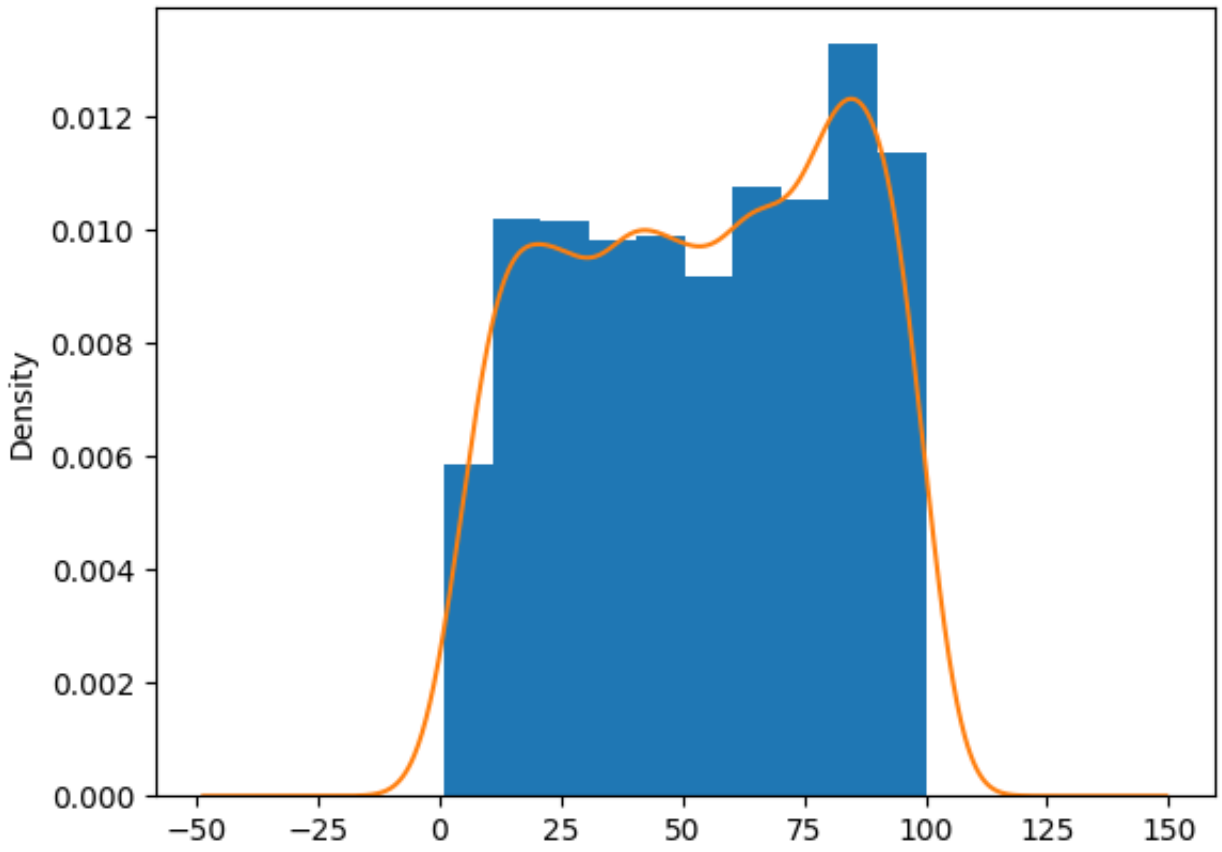
```
<Axes: xlabel='IMDB_Rating', ylabel='Count'>
```



Q: Can you plot the histogram and KDE of the `Rotten_Tomatoes_Rating`?

```
# YOUR SOLUTION HERE
movies['Rotten_Tomatoes_Rating'].hist(bins=10, density=True)
movies['Rotten_Tomatoes_Rating'].plot(kind='kde')
```

```
<Axes: ylabel='Density'>
```

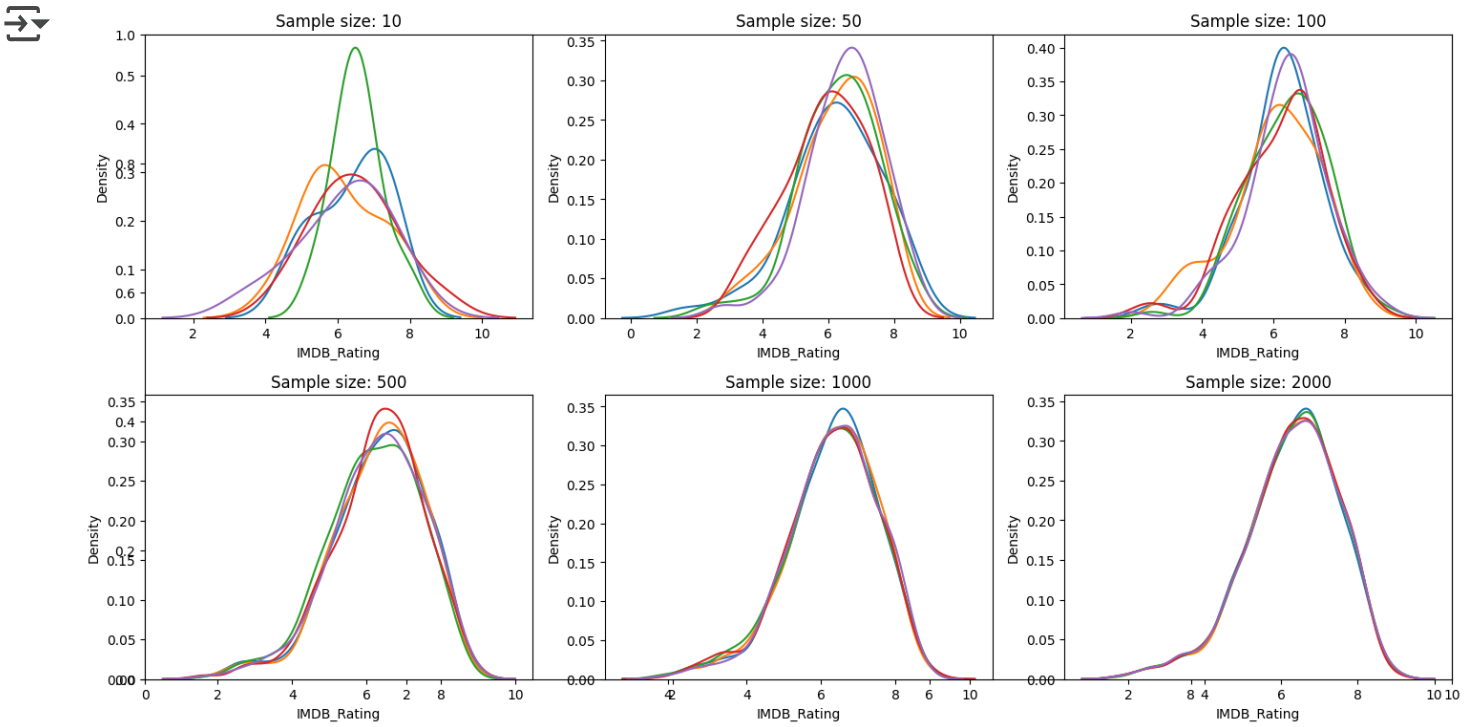


Let's study how KDE changes with the size of the data.

We can get a random sample using the pandas' `sample()` function. The `kdeplot()` function in seaborn provides many options (like kernel types) to do KDE. Let's sample some data points and see how does KDE plot changes with the size of the samples.

```
f = plt.figure(figsize=(15,8))
plt.xlim(0, 10)

sample_sizes = [10, 50, 100, 500, 1000, 2000]
for i, N in enumerate(sample_sizes, 1):
    plt.subplot(2,3,i)
    plt.title("Sample size: {}".format(N))
    for j in range(5):
        s = movies['IMDB_Rating'].sample(N)
        sns.kdeplot(s, legend=False)
plt.tight_layout()
```



We can also draw KDE plots using scikit-learn to change kernel functions.

First, we need points to score across.

- Remember the np.linspace() function?
- IMDB scores are only between 1 and 10. Let's create 1000 points between 1 and 10.

```
# use the linespace function to create a list of 1000 values
# between 1 and 10
# Store the sequence in a variable called X_vals

# YOUR SOLUTION HERE


X_vals = np.linspace(1, 10, 1000)
```

```
X_vals.shape
```

```
(1000,)
```

Our array needs to have 2 dimensions for score_samples() to work. Let's add another dimension to our array using np.newaxis.

```
X_plot = X_vals[:, np.newaxis]
X_plot[0:10]
```

```
array([[1.        ],
       [1.00900901],
       [1.01801802],
       [1.02702703],
       [1.03603604],
       [1.04504505],
       [1.05405405],
       [1.06306306],
       [1.07207207],
       [1.08108108]])
```

```
X_plot.shape
```

```
(1000, 1)
```

The KernelDensity function in scikit-learn needs a 2d array for the samples when we fit to the different kernels. Multidimensional scaling is deprecated for pandas series. We can cast the series to a numpy array before adding another dimension to solve this issue.

```
sample = movies['IMDB_Rating'].sample(5)
print(sample)
print(type(sample))
print(sample.shape)
```

```
2490    5.8
434     8.0
3040    6.4
2943    7.3
2353    5.9
Name: IMDB_Rating, dtype: float64
<class 'pandas.core.series.Series'>
(5,)
```

```
sample = np.array(sample)
print(sample)
print(type(sample))
print(sample.shape)
```

```
[6.5 5.4 6.3 ... 6.  8.  4.8]
<class 'numpy.ndarray'>
(2000,)
```

```
# Add another dimension to sample using np.newaxis
# Just like we did to X_vals.

# YOUR SOLUTION HERE

sample = sample[:, np.newaxis]
```

Now, let's plot using the **`tophat`** and **`gaussian`** kernels using the different sample sizes we used before. We'll do this using scikit-learn's `KernelDensity()` function.

```
from sklearn.neighbors import KernelDensity

# Step 1: Create values between 1 and 10 to evaluate the density
X_vals = np.linspace(1, 10, 1000)
X_plot = X_vals[:, np.newaxis]

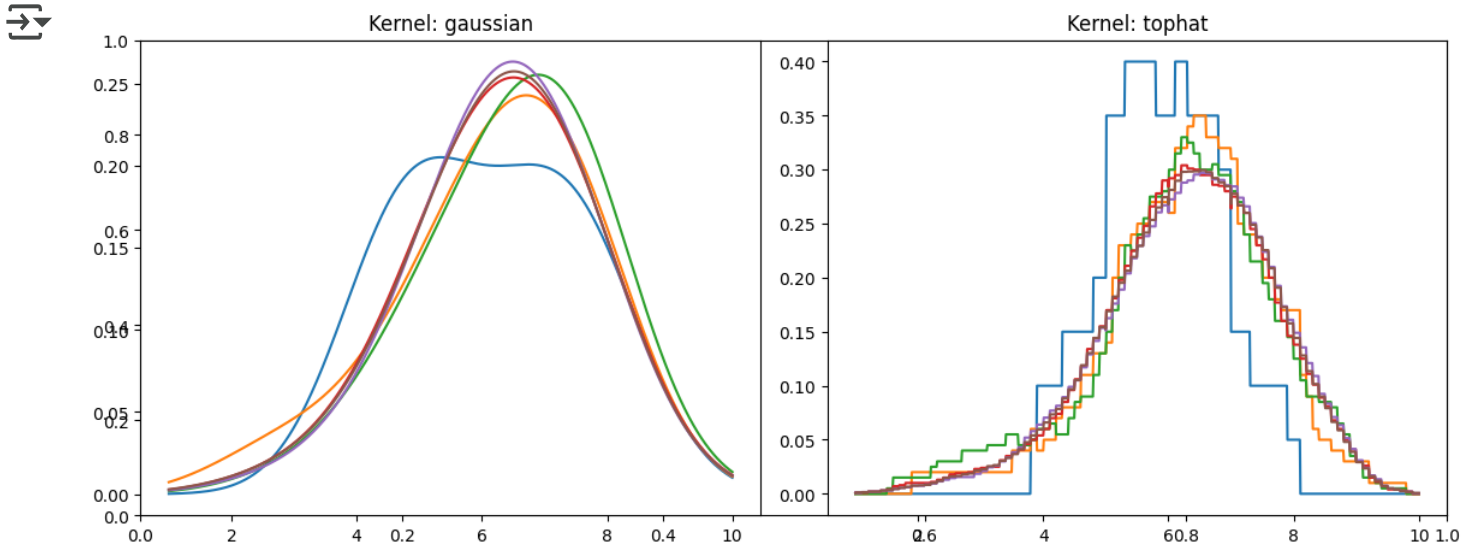
# Step 2: Define a function to plot KDE with a given kernel
def plot_kde(X_plot, sample_sizes, movies, kernel='gaussian'):
    for sample_size in sample_sizes:
        sample = np.array(movies['IMDB_Rating'].sample(sample_size))[:, np.newaxi
        kde = KernelDensity(kernel=kernel).fit(sample)
        log_density = kde.score_samples(X_plot)
        plt.plot(X_plot[:, 0], np.exp(log_density), '-')

# Step 3: Plot for both 'gaussian' and 'tophat' kernels
sample_sizes = [10, 50, 100, 500, 1000, 2000]
fig, ax = plt.subplots(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.title("Kernel: gaussian")
plot_kde(X_plot, sample_sizes, movies, kernel='gaussian')

plt.subplot(1, 2, 2)
plt.title("Kernel: tophat")
plot_kde(X_plot, sample_sizes, movies, kernel='tophat')

plt.tight_layout()
```



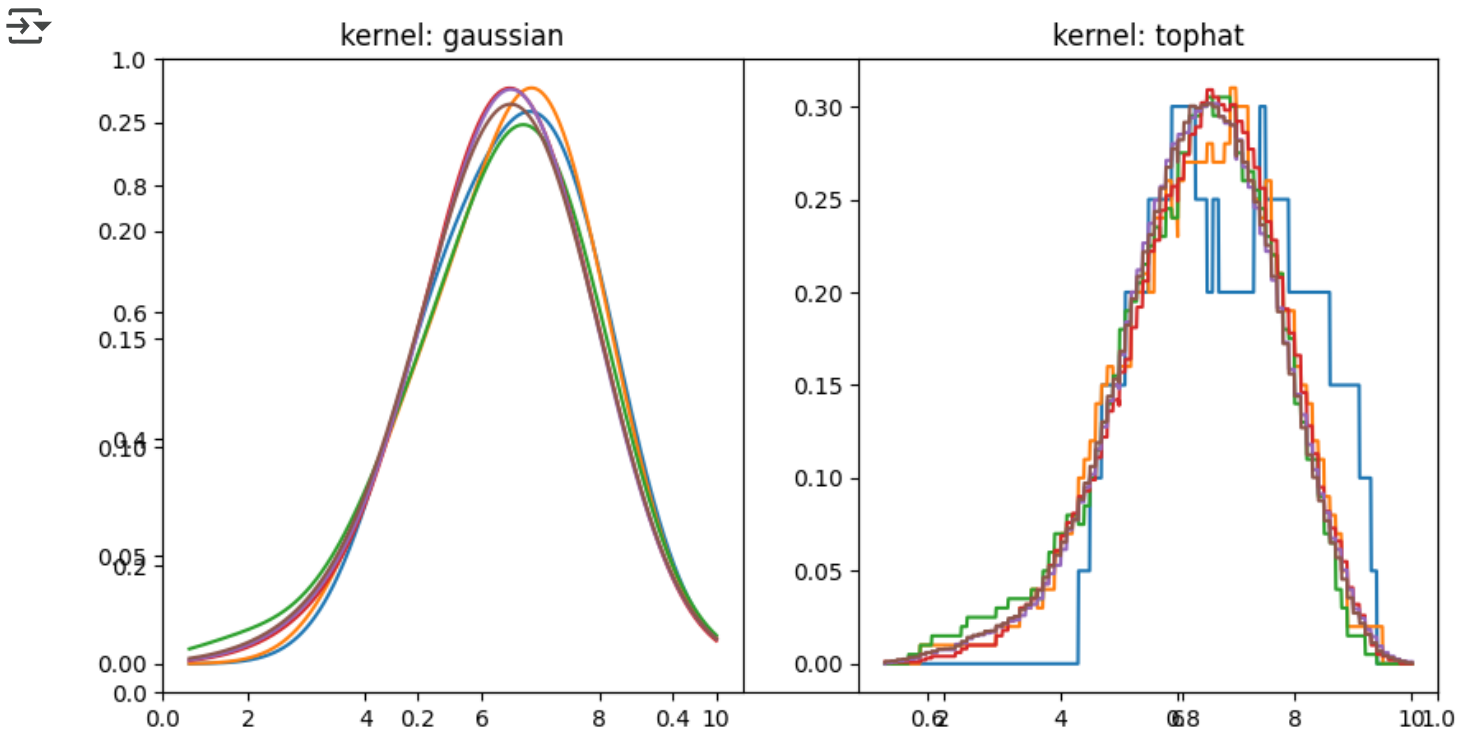

```
from sklearn.neighbors import KernelDensity
sample_sizes = [10, 50, 100, 500, 1000, 2000]

def plot_kde(X_plot, sample_sizes, movies, kernel='gaussian'):
    for sample_size in sample_sizes:
        sample = np.array(movies['IMDB_Rating'].sample(sample_size))
        sample = sample[:, np.newaxis]
        kde = KernelDensity(kernel=kernel).fit(sample)
        log_density = kde.score_samples(X_plot)
        plt.plot(X_plot[:, 0], np.exp(log_density), '-')

fig, ax = plt.subplots(figsize=(10,5))

# Let's look at the gaussian kernel like before
# put it in the first spot on a 1x2 grid
plt.subplot(1,2,1)
plt.title("kernel: {}".format('gaussian'))
plot_kde(X_plot, sample_sizes, movies)
# Now let's look at the tophat kernel
# put it in the second spot on a 1x2 grid
plt.subplot(1,2,2)
plt.title("kernel: {}".format('tophat'))
```

```
plot_kde(X_plot, sample_sizes, movies, kernel='tophat')
```



Let's try all kernel types using scikit-learn's `KernelDensity()`. Plot a 2x3 grid, however, instead of changing sample sizes like above, make each plot use the 6 different kernels supported by scikit-learn. Keep the sample size to 2 so we can more easily see the different distribution shapes. Also, draw a different sample (of 2) 5 times like the plots we drew with seaborn above.

Helpful links:

- https://scikit-learn.org/stable/modules/density.html
- https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KernelDensity.html#sklearn.neighbors.KernelDensity
- https://scikit-learn.org/stable/auto_examples/neighbors/plot_kde_1d.html

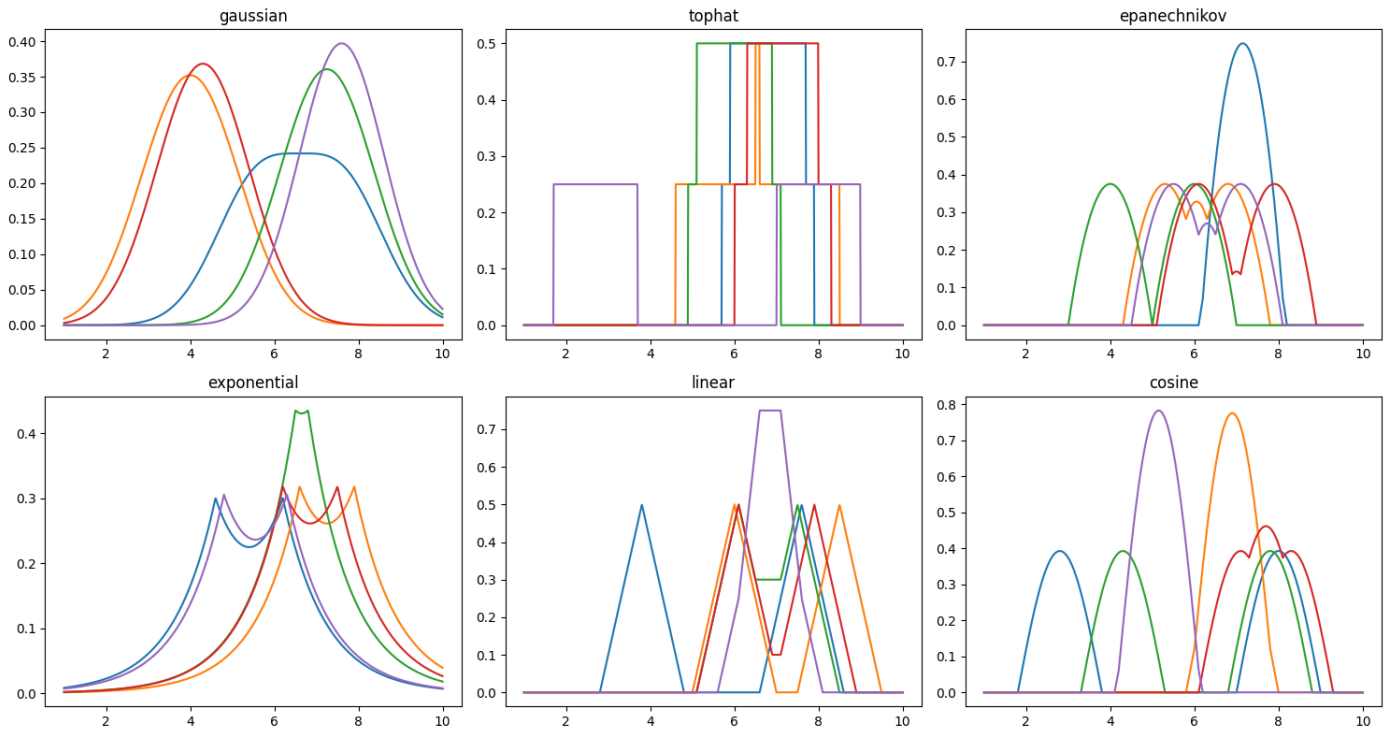
```
# YOUR SOLUTION HERE

from sklearn.neighbors import KernelDensity

kernels = ['gaussian', 'tophat', 'epanechnikov', 'exponential', 'linear', 'cosine
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for ax, kernel in zip(axes.ravel(), kernels):
    for _ in range(5):
        sample = np.array(movies['IMDB_Rating'].sample(2))[:, np.newaxis]
        kde = KernelDensity(kernel=kernel).fit(sample)
        log_dens = kde.score_samples(X_plot)
        ax.plot(X_plot[:, 0], np.exp(log_dens))
        ax.set_title(kernel)

plt.tight_layout()
```



Q: Let's use Seaborn again. We can play with the bandwidth option. Make sure to set the `xlim` so that all plots have the same x range, so that we can compare.

```
f = plt.figure(figsize=(15,8))
bw = ['scott', 'silverman', 0.01, 0.1, 1, 5]
sample_size = 10

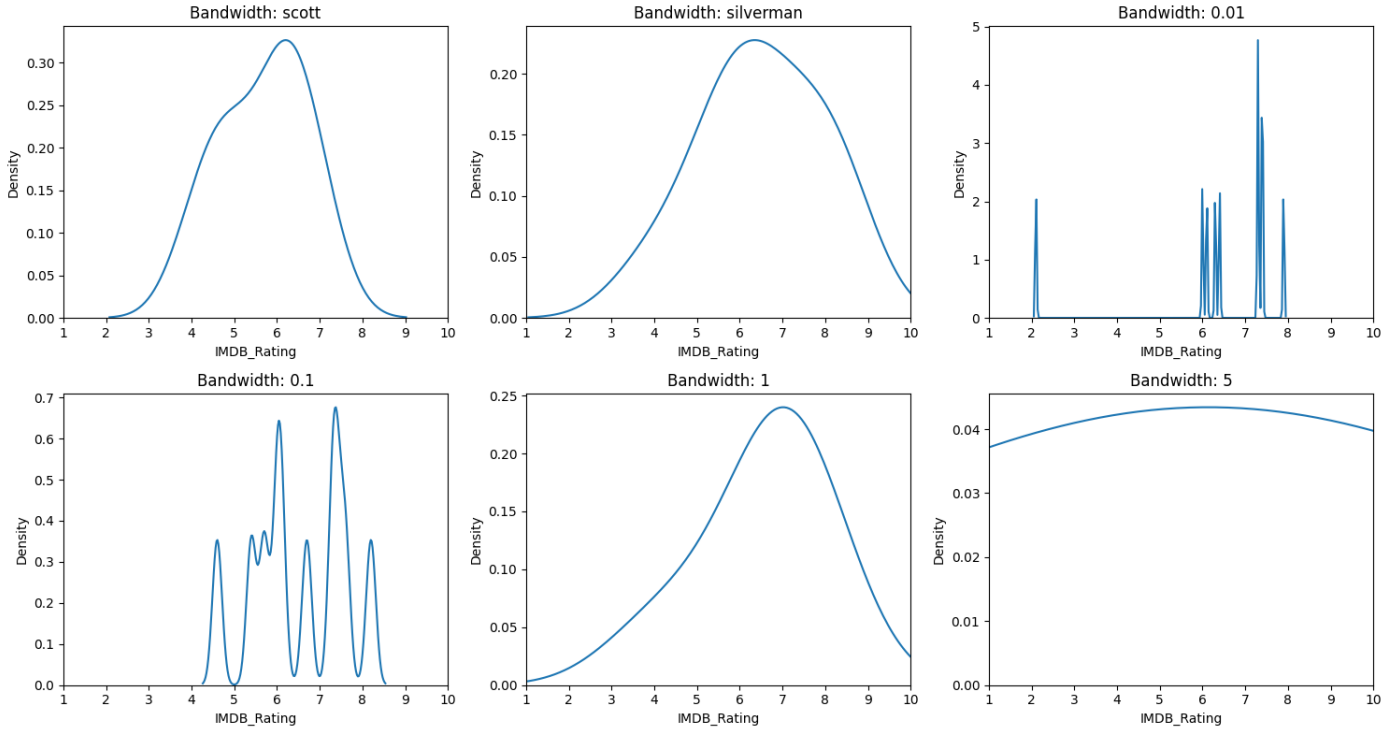
# YOUR SOLUTION HERE
```

```
f = plt.figure(figsize=(15, 8))
bw = ['scott', 'silverman', 0.01, 0.1, 1, 5]
sample_size = 10
```

```
for i, b in enumerate(bw, 1):
    plt.subplot(2, 3, i)
    sns.kdeplot(movies['IMDB_Rating'].sample(sample_size), bw_method=b)
    plt.title(f"Bandwidth: {b}")
    plt.xlim(1, 10)

plt.tight_layout()
```

```
<Figure size 1500x800 with 0 Axes>
```



Q: What's your takeaway? Explain how bandwidth affects the result of your visualization.

```
# YOUR SOLUTION HERE
```

Bandwidth controls the smoothness of the KDE curve:

- **Small bandwidths** (like 0.01 or 0.1) make the KDE very sensitive to individual data points. The plot may look spiky or noisy.
- **Large bandwidths** (like 1 or 5) smooth out the curve too much, which can hide important features or peaks.
- Automatic options like **`scott`** and **`silverman`** provide balanced defaults, but still may not be ideal in every case.

Choosing the right bandwidth is important — it affects how well you can see the overall pattern in the data without losing useful detail or adding too much noise.

Interpolation

One area where interpolation is used a lot is image processing. Play with it!

https://matplotlib.org/examples/images_contours_and_fields/interpolation_methods.html

[ ] ↳ 22 cells hidden

Moving average

Pandas has a nice method called `rolling()`: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.rolling.html

It lets you do operations on the `rolling` windows. For instance, if you want to calculate the moving average, you can simply

[ ] ↳ 3 cells hidden

Examining relationsips

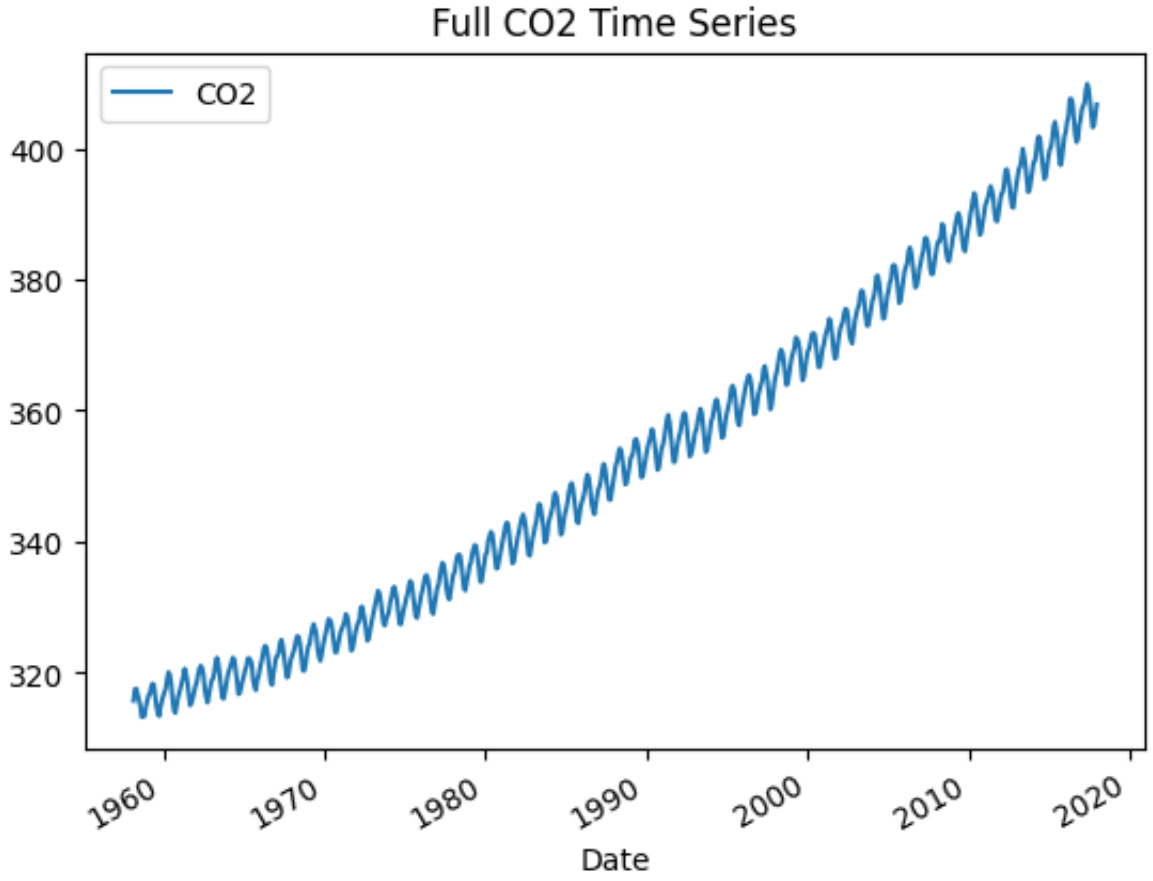
Remember Anscombe's quartet? Actually, the dataset is not only included in `vega_datasets` but also in `seaborn`.

```
# Load the CO2 data and convert the date column
co2 = vega_datasets.data.co2_concentration()
co2['Date'] = pd.to_datetime(co2['Date'])

# Set Date as index
co2.set_index('Date', inplace=True)

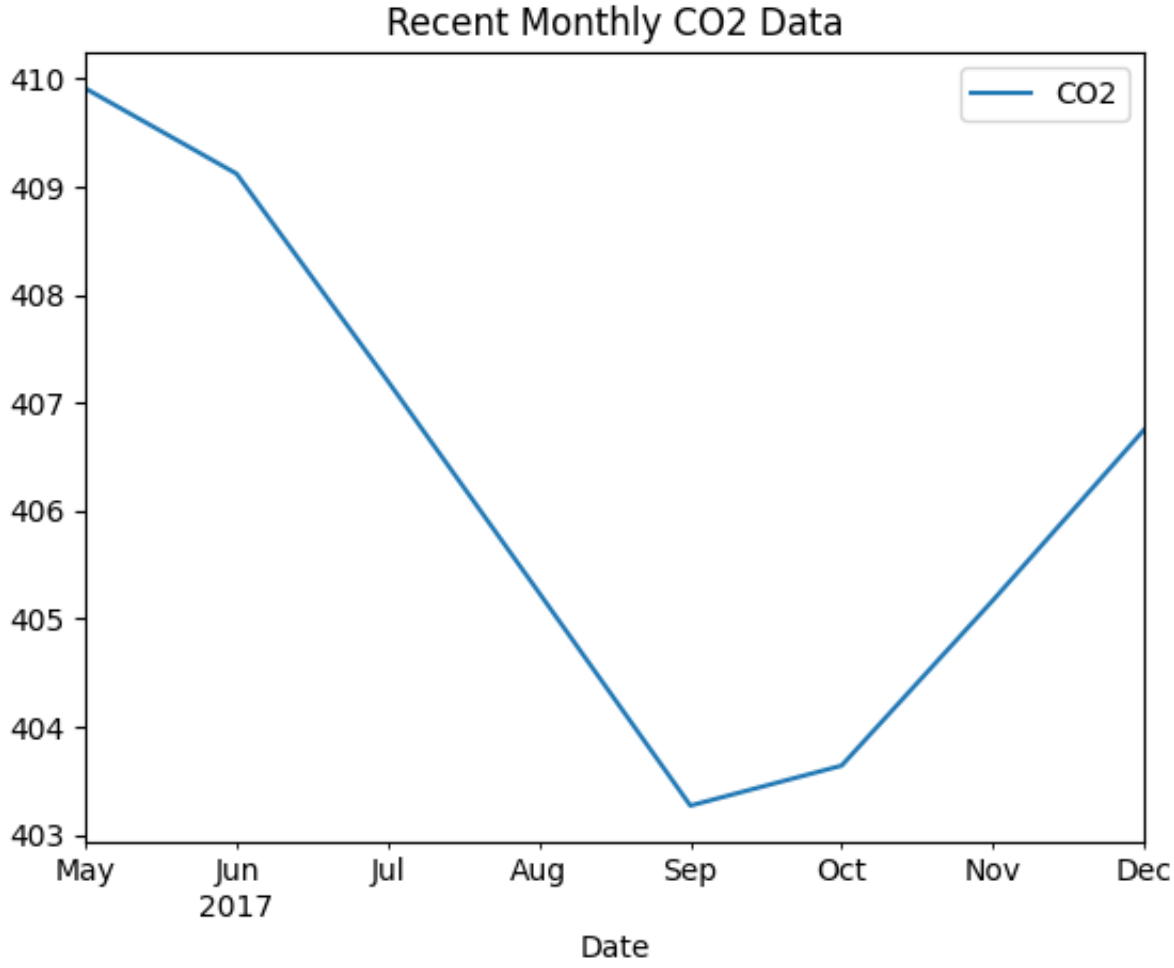
# Plot full data
co2.plot(title='Full CO2 Time Series')
```

```
<Axes: title={'center': 'Full CO2 Time Series'}, xlabel='Date'>
```



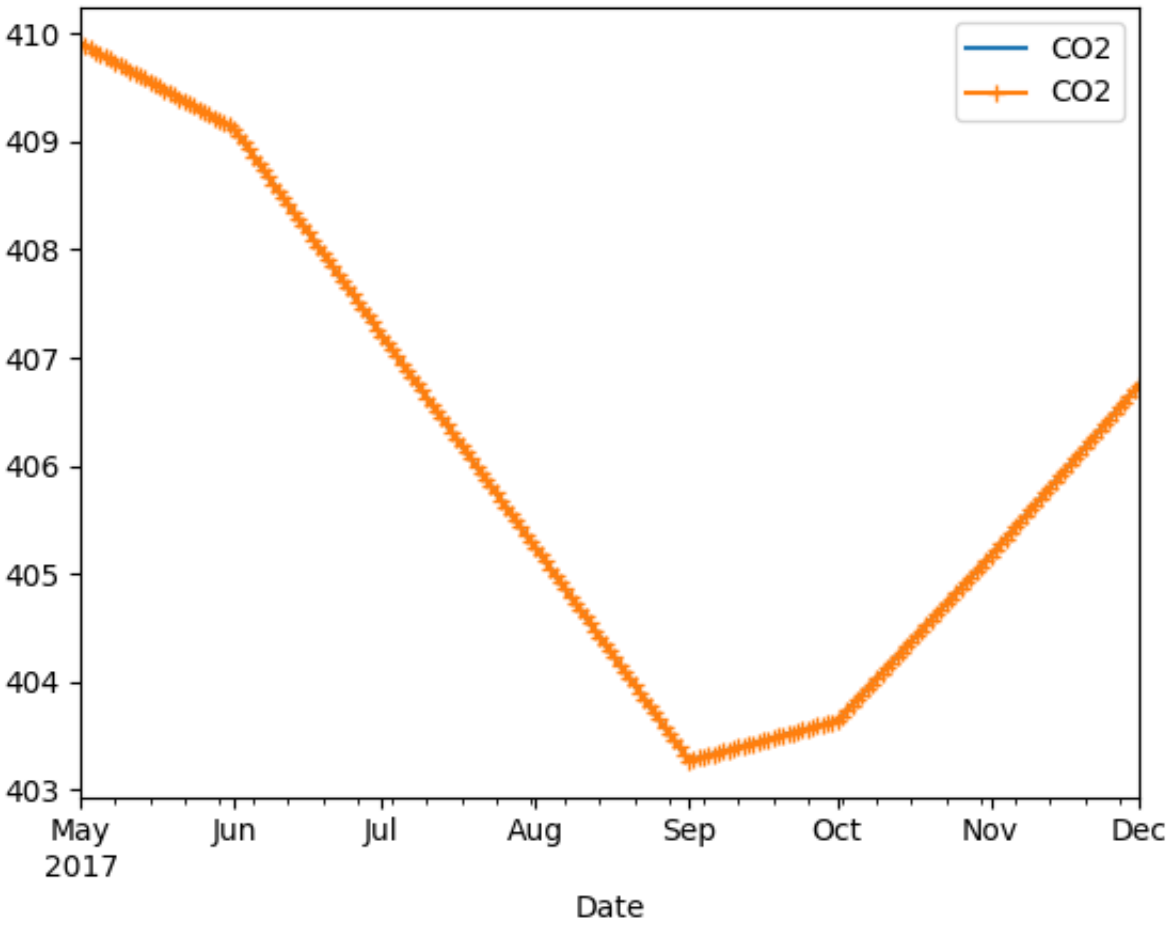
```
recent_co2 = co2.tail(8)
recent_co2.plot(title='Recent Monthly CO2 Data')
```

```
<Axes: title={'center': 'Recent Monthly CO2 Data'}, xlabel='Date'>
```



```
upsampled = recent_co2.resample('D').interpolate(method='linear')
ax = recent_co2.plot(style='o', title='Linear Interpolation (Daily)', ms=6)
upsampled.plot(ax=ax, style='-+')
```

```
<Axes: xlabel='Date'>
```



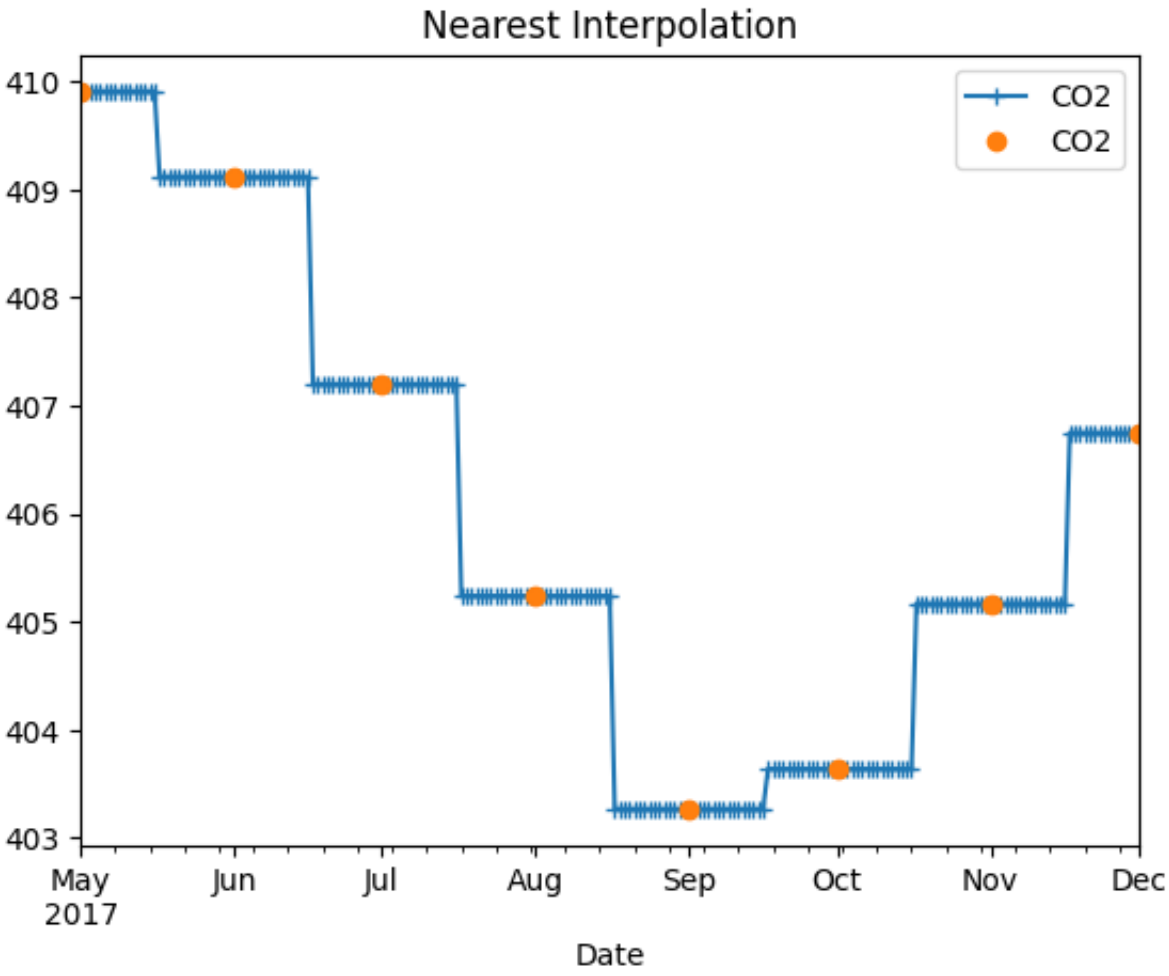
```
df = sns.load_dataset("anscombe")
df.head()
```

	dataset	x	y
0	I	10.0	8.04
1	I	8.0	6.95
2	I	13.0	7.58
3	I	9.0	8.81
4	I	11.0	8.33

Next steps: Generate code with df | View recommended plots | New interactive sheet

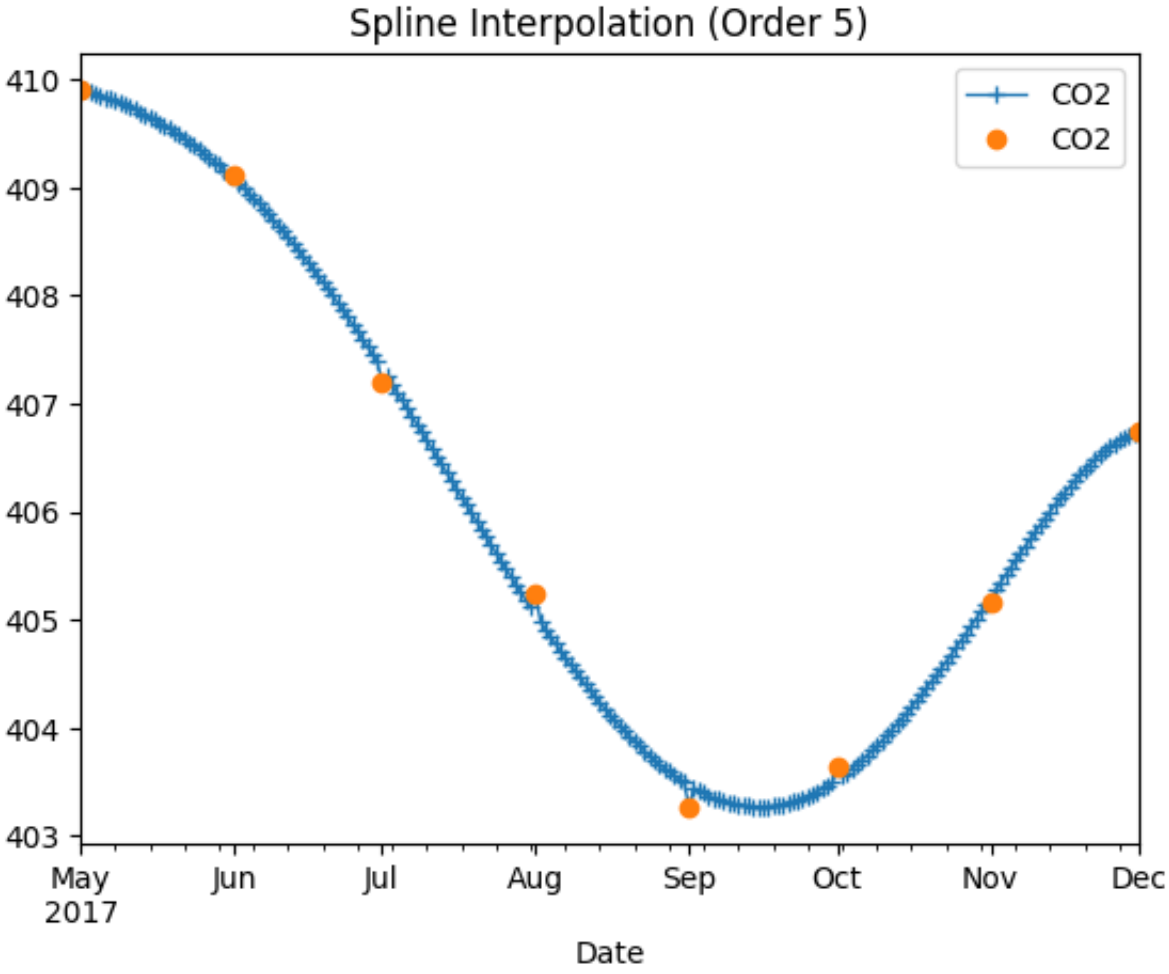
```
# Nearest interpolation
ax = recent_co2.resample('D').interpolate(method='nearest').plot(
    style='+-', title='Nearest Interpolation')
recent_co2.plot(ax=ax, style='o', ms=6)
```

```
<Axes: title={'center': 'Nearest Interpolation'}, xlabel='Date'>
```



```
# Spline interpolation (order 5)
ax = recent_co2.resample('D').interpolate(method='spline', order=5).plot(
    style='+-', lw=1, title='Spline Interpolation (Order 5)')
recent_co2.plot(ax=ax, style='o', ms=6)
```

```
<Axes: title={'center': 'Spline Interpolation (Order 5)'}, xlabel='Date'>
```



All four datasets are in this single data frame and the 'dataset' indicator is one of the columns. This is a form often called tidy data, which is easy to manipulate and plot. In tidy data, each row is an observation and columns are the properties of the observation. Seaborn makes use of the tidy form. Using seaborn's `lmplot`, you can very quickly examine relationships between variables, separated by some facets of the dataset.

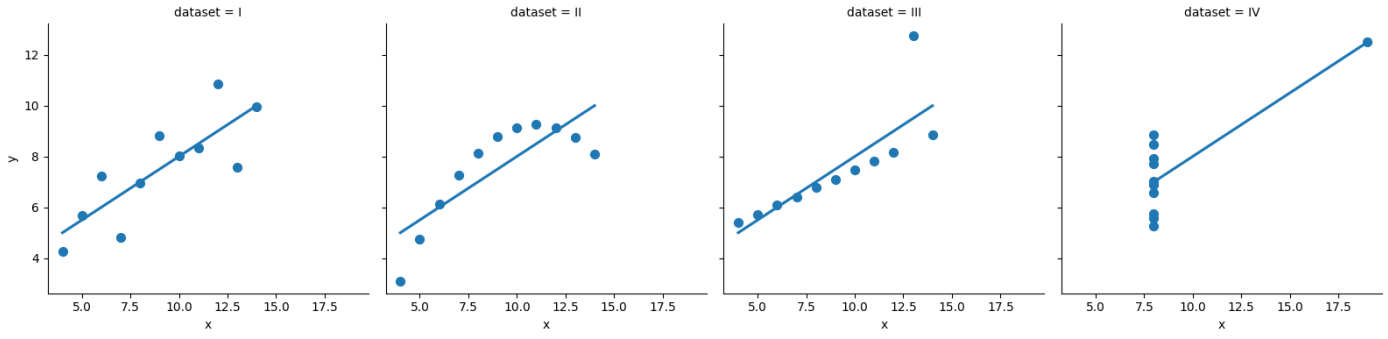
Q: Can you produce the plot below using `lmplot()`?

```
# plotting parameters you can use
palette = "muted"
scatter_kws={"s": 50, "alpha": 1}
ci=None
height=4

# Implement

# YOUR SOLUTION HERE
sns.lmplot(
    x='x', y='y', col='dataset',
    data=df, ci=None,
    palette='muted', height=4,
    scatter_kws={"s": 50, "alpha": 1}
)
```

```
<seaborn.axisgrid.FacetGrid at 0x7c2fb89f0e50>
```

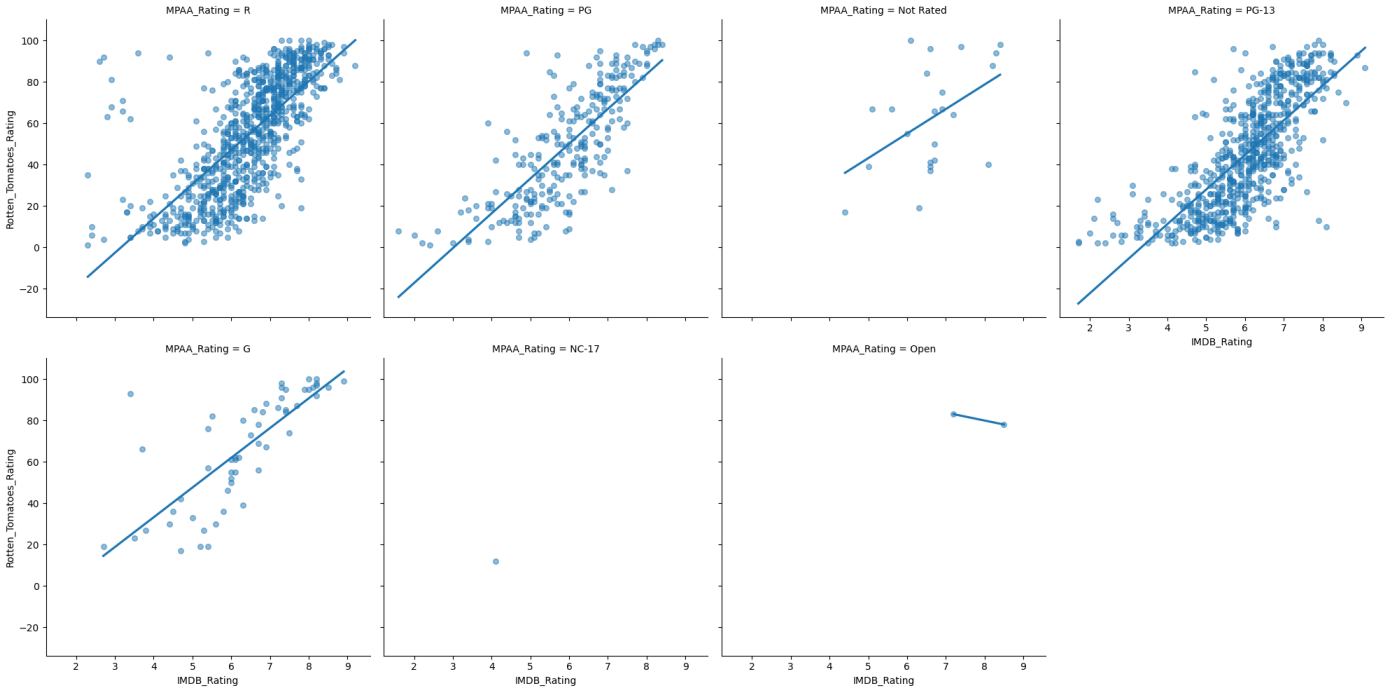


Q: So let's look at the relationship between `IMDB_Rating` and `Rotten_Tomatoes_Rating` in the `movies` dataset, separated with respect to `MPAA_Rating`. Put 4 plots in a row.

```
# YOUR SOLUTION HERE

sns.lmplot(
    x='IMDB_Rating', y='Rotten_Tomatoes_Rating', col='MPAA_Rating',
    data=movies, col_wrap=4, ci=None,
    palette='muted',
    scatter_kws={'s': 30, 'alpha': 0.5}
)
```

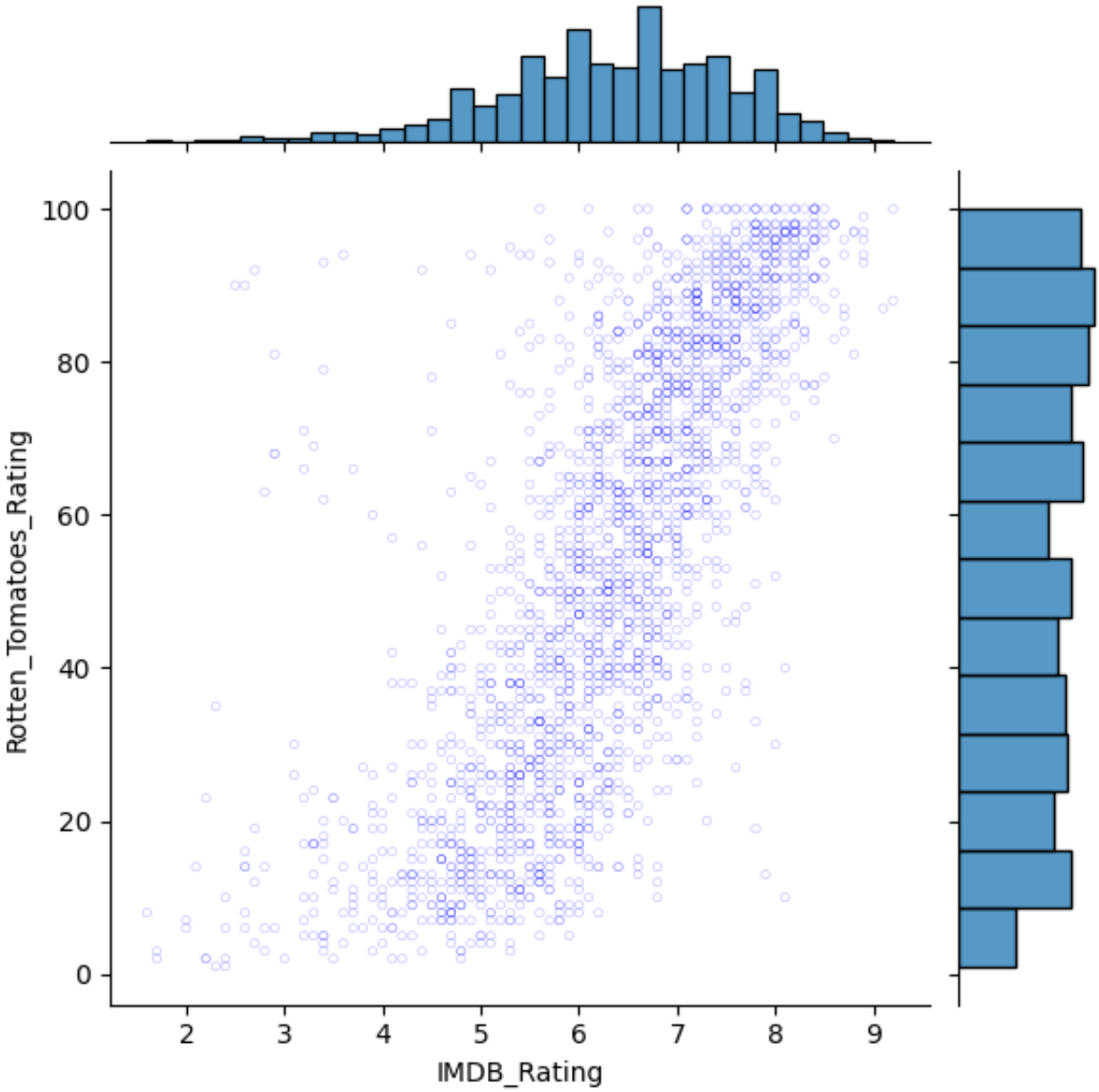
```
<seaborn.axisgrid.FacetGrid at 0x7c2fb87f9f10>
```



It may be interesting to dig up what are the movies that have super high Rotten Tomatoes rating and super low IMDB rating (and vice versa)!

Another useful method for examining relationships is **`jointplot()`**, which produces a scatter plot with two marginal histograms.

```
g = sns.jointplot(x = movies['IMDB_Rating'], y = movies['Rotten_Tomatoes_Rating']
```



Hexbin density plot

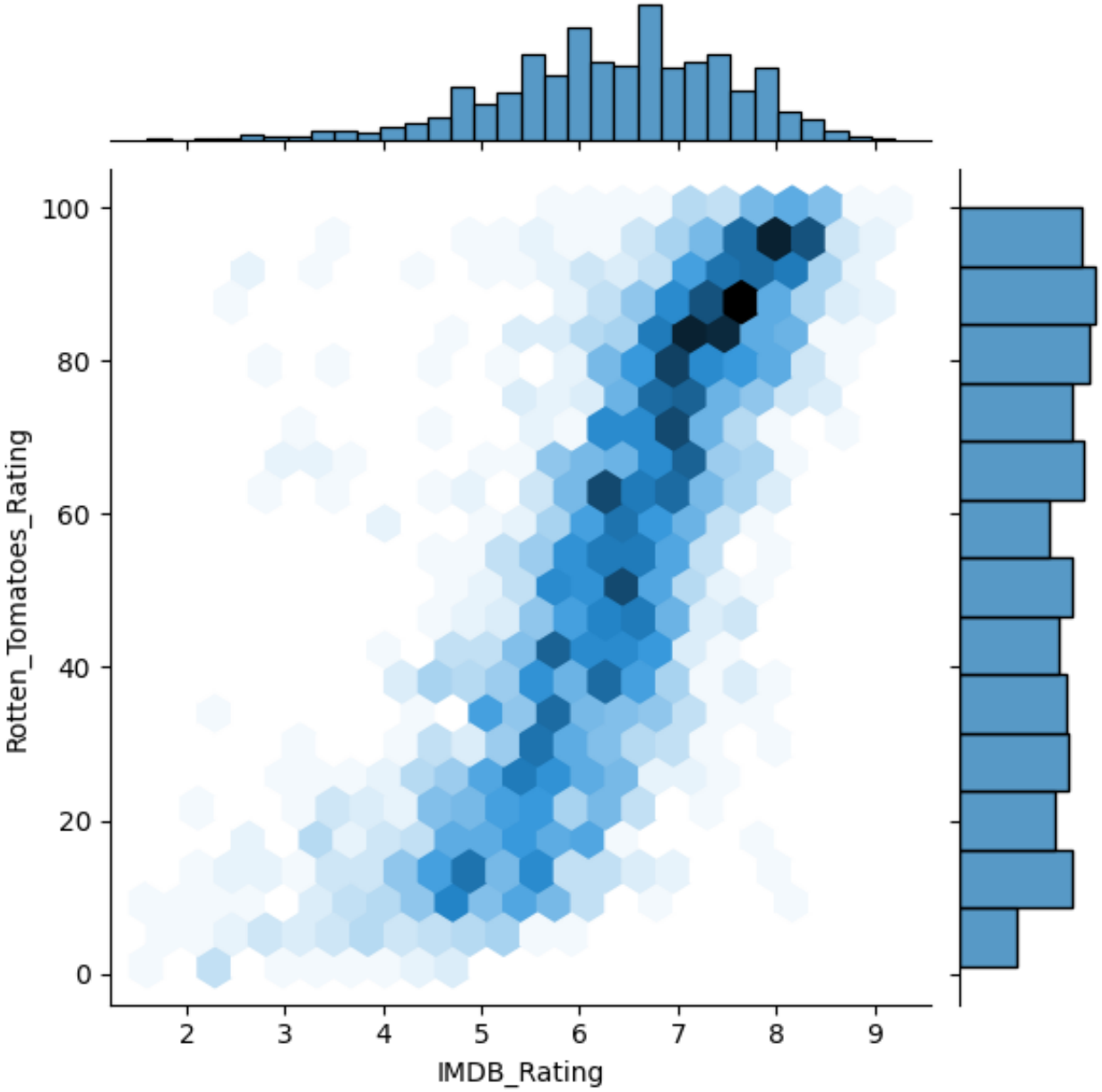
In 2D, *heatmap* can be considered as a color-based histogram. You divide the space into bins and show the frequency with colors. A common binning method is the hexagonal bin.

We can again use the `jointplot()` and setting the `kind` to be `hexbin`.

Q: Can you create one?

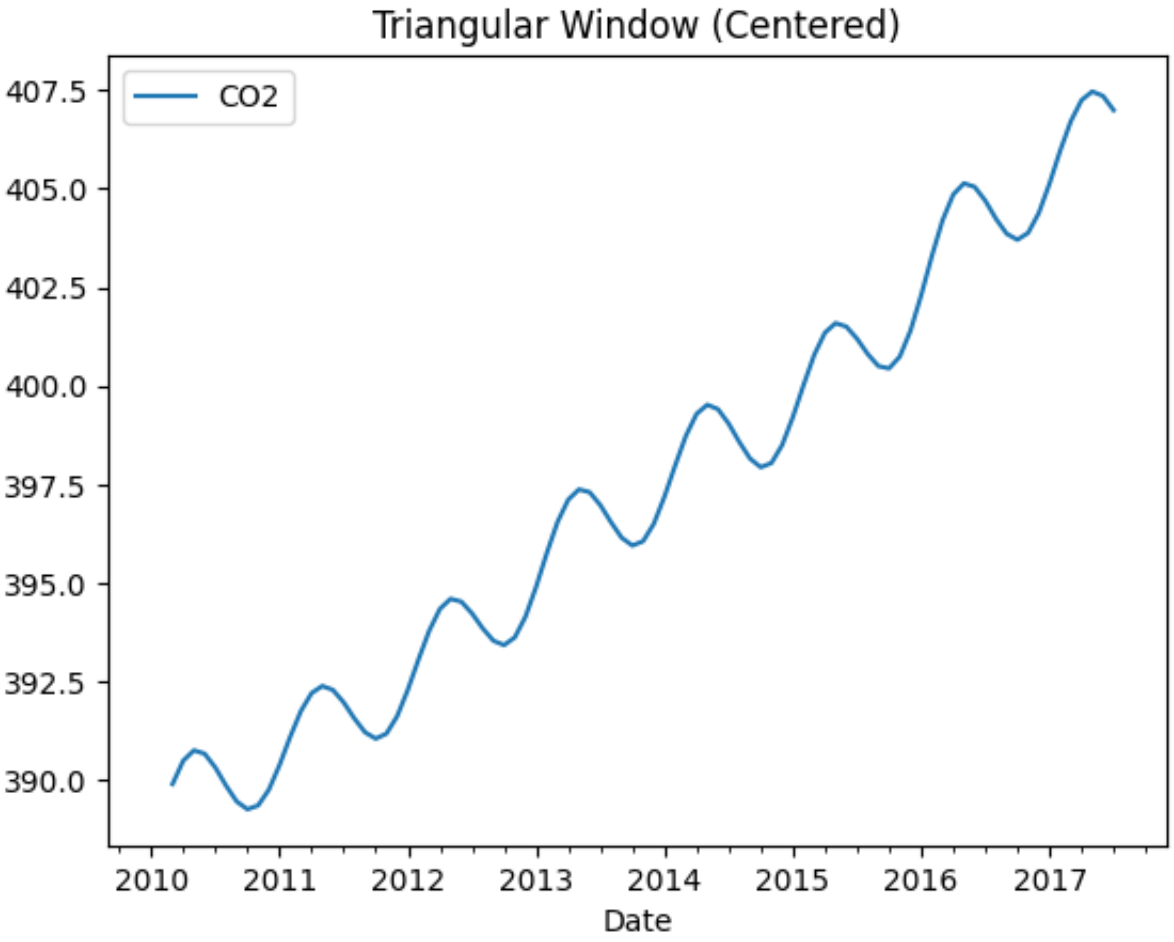
```
# YOUR SOLUTION HERE
# Hexbin
sns.jointplot(
    x='IMDB_Rating', y='Rotten_Tomatoes_Rating',
    data=movies, kind='hex'
)
```

```
<seaborn.axisgrid.JointGrid at 0x7c2fb92dc5d0>
```



```
co2[-100:].rolling(12, win_type='triang', center=True).mean().plot(title='Triangula
```

```
<Axes: title={'center': 'Triangular Window (Centered)'}, xlabel='Date'>
```



2D KDE

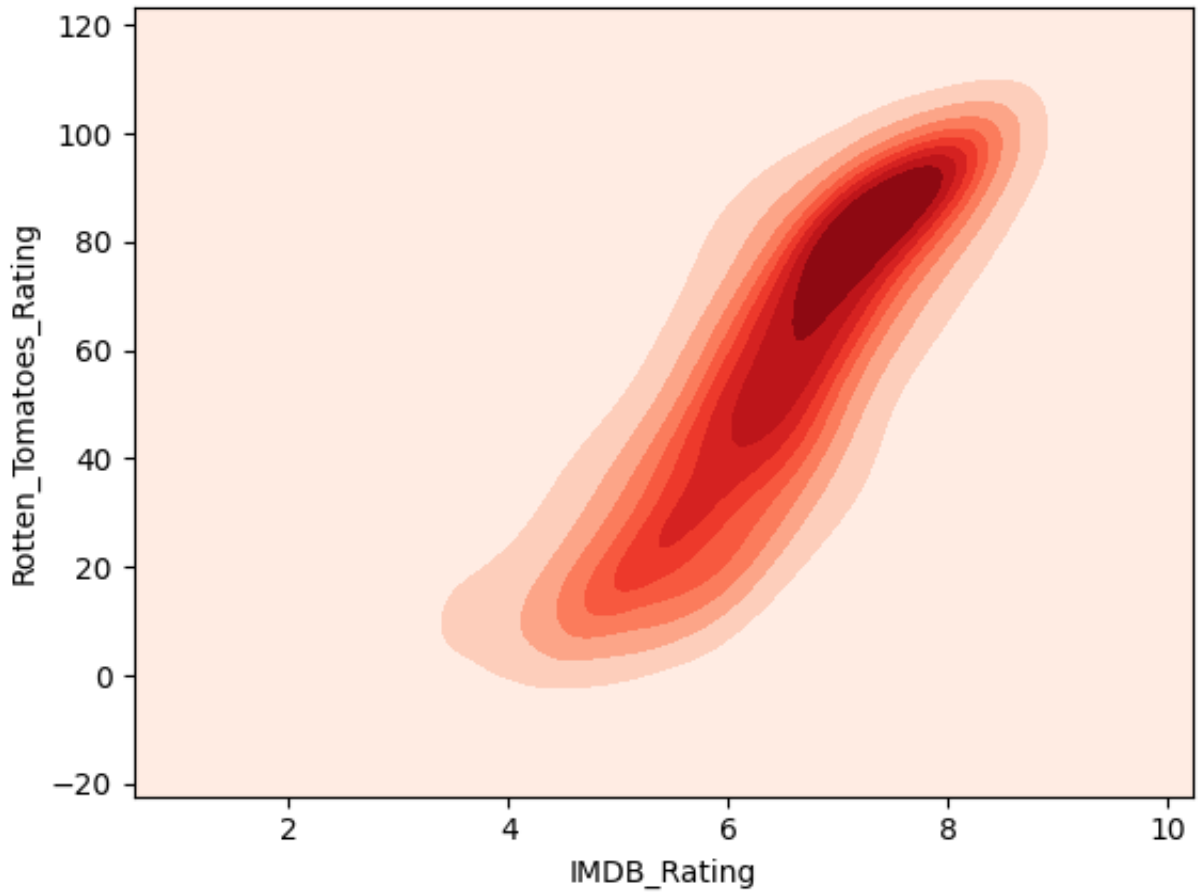
We can also do 2D KDE using seaborn's **kdeplot()** function.

Q: Can you draw one like this?

```
cmap = "Reds"
fill = True         # what happens if you change this?
thresh = 0        # what happens if you change this?

# YOUR SOLUTION HERE
# 2D KDE
sns.kdeplot(
    x=movies['IMDB_Rating'],
    y=movies['Rotten_Tomatoes_Rating'],
    cmap="Reds", fill=True, thresh=0
)
```

```
<Axes: xlabel='IMDB_Rating', ylabel='Rotten_Tomatoes_Rating'>
```



Or again using **`jointplot()`** by setting the `kind` parameter. Look, we also have the 1D marginal KDE plots!

Q: create jointplot with KDE

Note that the X-axis is logarithm of `IMDB_Votes`.

```
x = np.log(movies['IMDB_Votes'])

# YOUR SOLUTION HERE.
# KDE with log-scaled IMDB_Votes
x = np.log(movies['IMDB_Votes'])
sns.jointplot(
    x=x, y=movies['Rotten_Tomatoes_Rating'],
    kind='kde'
)
```

```
<seaborn.axisgrid.JointGrid at 0x7c2fb93b6550>
```

